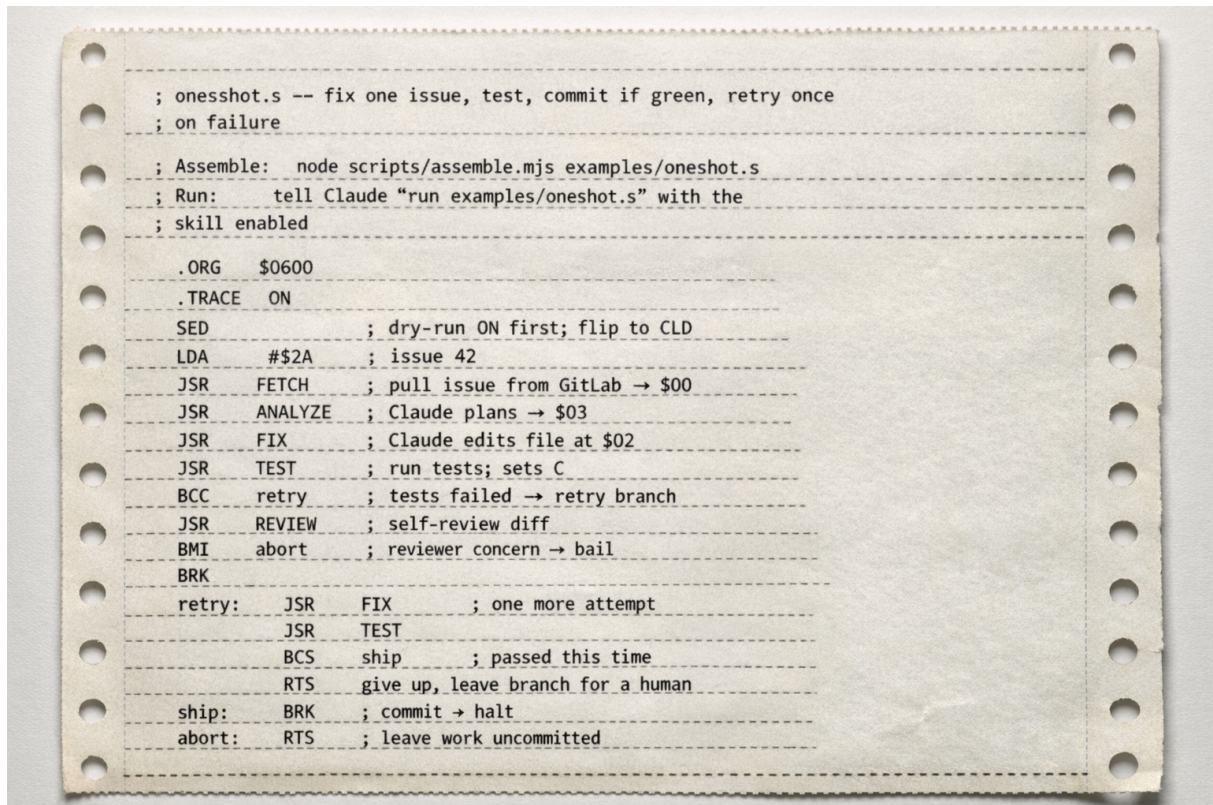


I Programmed an AI in 6502 Assembly. It Worked.

A tongue-in-cheek Claude Code skill that uses 1975-era mnemonics to drive a modern AI and accidentally saves a pile of tokens doing it.

Paul Newell · Apr 2026 · 8 min read



The oneshot.s program. Looking exactly how it should.

I'll be honest, this started as a joke. I wanted to see if I could get Claude Code to respond entirely in 6502 assembly language. The 6502 powered the Apple II, the Commodore 64, the BBC Micro, the NES. It had 56 opcodes, 64KB of address space and absolutely no business being anywhere near a large language model.

So obviously I built a Claude Code skill that maps its instruction set onto a modern triage-and-fix workflow. You write `.s` files. Claude executes them. Its response is also assembly. There's a zero-page. There's a stack. `BRK` means "commit and halt."

And then something unexpected happened. It started saving tokens. A lot of them.

Opcode Oriented Programming (OOP) :D

6502 mnemonics. Modern AI execution. Fewer tokens.

The Punchline That Became the Point

Claude's output (the expensive side of the API pipe) dropped by roughly 5-8x in triage sessions. An 800-token prose response ("Let me analyze this issue. I'll start by examining the file at...") became a handful of lines of assembly trace. Issue fetched, file fixed, tests passed, diff reviewed, committed, PR opened. No "Let me now...", no "I'll proceed to...", no "Here's what I did...". Just the trace.

Why does this save tokens? The savings come almost entirely from constraining Claude's *output* format. Writing JSR FIX instead of "please fix it" in the input is basically a wash. The win is that Claude's reply is structured assembly and BPE tokenization rewards short repetitive output. Sessions with lots of tool calls save the most because prose Claude narrates every single step. Opcode Claude just emits one trace line per instruction and moves on.

How It Works: 15 Opcodes and 9 Vectors

The skill is called **opcode** and it defines a semantic DSL that borrows the 6502's mnemonics but maps them onto issue triage. The core ISA is just 15 opcodes. You can genuinely hold the entire thing in your head.

LDA loads an issue. STA persists to a slot. LDX / INX / CPX handle loops. JSR calls one of nine I/O vectors: FETCH, FIX, TEST, LINT, REVIEW, PUSH, PULL, CLONE, ANALYZE. BCC / BCS branch on test results. BRK commits and halts. PHA / PLA push and pop todos from a stack that maps directly to Claude Code's task list.

There's a zero-page with named slots. \$00 is the current issue ID, \$01 is the branch name, \$02 is the current file, \$05 is the commit message. The fetched issue queue lives at \$10-\$1F. The stack page at \$0100-\$01FF is the Claude Code task list.

It talks to GitHub or GitLab through a forge abstraction layer so JSR PUSH opens a PR on GitHub or an MR on GitLab. Nobody has to compromise their vocabulary.

. . .

The Smallest Useful Program

Six opcodes, three flags in play. This is the heart of what the whole thing does:

```
.ORG    $0600
LDA     #$2A      ; issue 42, Hex is not mandatory
JSR     FETCH
JSR     FIX
JSR     TEST
BCC     skip      ; tests failed -> bail
BRK                     ; commit + halt
skip:   RTS        ; leave uncommitted
```

If the tests pass (`C=1`), commit and halt. If they fail (`C=0`), branch to `skip` and return without committing. That's it. The `BCC / BCS` pattern carries over directly from the real 6502. Carry set means success, carry clear means failure.

A Real Triage Session

Here's `triage-bugs.s` which drains all bug-labeled issues using the todo stack:

```

.ORG    $0600

LDA     #$01      ; label mask: bug
STA     $20       ; set label filter
JSR     FETCH     ; populate $10-$1F

; Push fetched issues onto the todo stack
LDX     #$00
push:   LDA     $10,X    ; load Xth issue
        BEQ     drain   ; empty -> drain
        PHA             ; push as todo
        INX
        CPX     #$10    ; queue bounds
        BNE     push

; Drain the stack: work each issue, skip on failure
drain:  PLA             ; pop newest -> A
        BEQ     done
        JSR     ANALYZE
        JSR     FIX
        JSR     TEST
        BCC     drain   ; failed -> skip
        JSR     REVIEW
        BCC     drain   ; concerns -> skip
        BRK             ; commit this one
        JMP     drain   ; keep going

done:   JSR     PUSH     ; open PRs
        RTS

```

This loops through every bug issue, tries to fix each one and only commits the ones where tests pass and the self-review is clean. Failed fixes get skipped. At the end it opens PRs for everything that worked. The `PHA / PLA` cycle maps directly onto Claude Code's task list so each pushed issue becomes a real tracked todo.



The program, in its preferred distribution format.

The Token Math

Here's a rough comparison of a typical multi-issue triage session. Three bugs fetched, two fixed successfully, one skipped:

	Prose Claude	Opcode Claude
Narration per tool call	~80 tokens	~15 tokens
Tool calls in session	12	12
Transition prose	~200 tokens	0
Summary at end	~150 tokens	0
Approximate total output	~1,300 tokens	~180 tokens

The tool calls themselves are identical. Same edits, same test runs, same git operations. The difference is everything *around* them. Prose Claude can't help explaining what it's about to do, what it just did and what it plans to do next. Opcode Claude just emits a trace line and cracks on.

The Constraint Is the Feature

There's a deeper thing going on here. The 6502 aesthetic isn't just nostalgic decoration, it's a forcing function. When your output format is assembly you commit to a verb sequence before execution instead of wandering through chat mode. The resulting traces scan way faster than prose. They're diffable, greppable and versionable. You can commit a `.s` file alongside your code and re-run it on the next batch of issues.

The temptation to add "just one sentence of context" is the whole failure mode. Resist it absolutely.

The skill is pretty militant about this. Claude's response must be a valid `.s` program. No greetings, no sign-offs, no "Let me", no markdown headers. If it doesn't fit as an instruction, a directive or a semicolon comment it doesn't get said. There are escape hatches like `.ASK` for questions, `.NOTE` for observations and `.ERR` for errors. But these are pressure-release valves, not invitations to prose.

The Extended ISA: For Completeness and Nostalgia

Because the joke demands it there's a full extended layer (~40 more mnemonics) you can opt into with `.EXTENDED ON`. Shifts become priority promotions (`ASL A` = "promote priority"). Logic ops become label-filter algebra (`AND #imm` = "intersect label mask"). `BVS` branches on merge conflict. Most of these are tagged as metaphor so they get narrated in the trace but not actually executed.

And because the joke *truly* demands it there are six illegal opcodes behind `.UNSAFE ON`. The real undocumented 6502 ops like `LAX`, `SAX` and `DCP`. Pure flavor. Narrated, never executed, not even with the directive set. They exist because "a museum exhibit behind glass" is funnier than not having them at all.

So Is This Actually Useful?

Genuinely, yes. If you run batch triage workflows like scanning a backlog, fixing straightforward bugs and opening PRs then the token savings are real and the structured output is way easier to read than prose. The `.s` files are reviewable artifacts. The traces are audit logs. And the constraint of planning your workflow as a sequence of opcodes before execution turns out to be surprisingly good discipline.

Is it the most practical way to interact with an AI? No. Would I recommend it for someone who just wants Claude to fix a bug? Also no. But if you're running the same triage pattern across many issues and you care about token costs and you happen to think "commit and halt" beats "I'll now commit these changes to the repository" then this might be your thing.

At the end of the day the question is the same one it's always been. Are you a quiche eater or a Real Programmer? Real Programmers don't need prose. They need opcodes, a carry flag and a `BRK` at the end.

Try it yourself: The skill is called **opcode**. Drop it into your `.claude/skills/` directory, point it at a repo with some open issues and run the oneshot example in dry-run mode. Every program ends the same way: `BRK`. Commit the work. Halt the machine.

Claude Code AI 6502 Assembly Developer Tools Retro Computing Token Optimization